

DigiZafe: Presentation & Viva Guide

This document is your master script, defense strategy, and study guide for presenting DigiZafe to a technical audience or professor.

PART 1 — 2-MINUTE EXPLANATION

"Good morning. My project is DigiZafe—a Personal Digital Exposure Intelligence & Remediation Platform. The core problem today is that people's personal data is scattered across data brokers and dark web breaches, and removing it manually is a legally complex, technically frustrating nightmare.

DigiZafe solves this by acting as both a secure search engine and a robotic privacy advocate. First, it uses asynchronous OSINT workers to scour the web and correlate discovered data back to the user via a deterministic, rule-based Identity Engine. Second, it calculates the user's exposure risk using a Machine Learning gradient boosting model. Finally—and this is the core innovation—it uses headless Chromium browser automation to physically visit data broker websites, navigate anti-bot protections, and submit automated privacy opt-out requests on the user's behalf. It's an end-to-end pipeline from discovery to actual remediation."

PART 2 — 5-MINUTE EXPLANATION

1. Problem: Data brokers profit off PII. Finding your exposure takes

hours; removing it takes weeks of manual form-filling. **2. Motivation:** Privacy should be accessible. The burden of executing data rights should be shifted from the human to automated agents. **3. Solution:** A distributed platform that finds the data, scores the risk, and removes it. **4. Architecture:** It's built on a modern, decoupled stack. The frontend is a React/Vite SPA. The backend is an asynchronous FastAPI Python server. Because OSINT scraping and web automation are highly latent, the API offloads all heavy lifting to a distributed cluster of Celery workers managed via Redis. All state is securely persisted in PostgreSQL. **5. Core Innovation:** The automated remediation engine. It doesn't just send emails; it utilizes Playwright to render real DOMs, inject CAPTCHA tokens, and submit complex JavaScript-heavy web forms entirely in the background. **6. Algorithms:** We use a deterministic Identity Match Engine to correlate OSINT data to users, ensuring legal explainability. For risk scoring, we train a `HistGradientBoostingRegressor` to predict the compounded danger of overlapping breaches. **7. Results:** The system successfully completes end-to-end flows: ingesting an email, querying HavelBeenPwned and surface web sources, calculating a risk score, and executing a dry-run Playwright remediation script. **8. Limitations:** The headless browser architecture is highly memory-intensive, making horizontal scaling expensive. Furthermore, heuristic DOM scraping is brittle—if a data broker changes their website's HTML, the script fails over to a 'Manual' fallback.

PART 3 — 15-MINUTE TECHNICAL PRESENTATION SCRIPT

(Slide 1: Title) "Welcome. Today I am presenting DigiZafe, an automated digital exposure and remediation platform."

(Slide 2: The Problem) "The internet is filled with our personal data. Finding it is hard. Removing it is harder. Data brokers intentionally build convoluted, CAPTCHA-heavy websites to prevent you from opting out. Our goal was to automate this entire lifecycle."

(Slide 3: Architecture Overview) "To achieve this, we needed an

architecture that could handle highly latent, unpredictable network tasks without blocking the user interface. We chose a React frontend communicating with a FastAPI backend. FastAPI is built on `asyncio`, making it perfect for concurrent I/O. For the heavy lifting, FastAPI delegates tasks to a Redis message broker, where Celery workers consume and execute them asynchronously. State is stored in PostgreSQL."

(Slide 4: Data Discovery & Correlation) "The first step is Discovery. Our Celery workers execute 'Connectors'—modules that query external OSINT sources like HavelBeenPwned. Because this data is messy, we store it in a Postgres `JSONB` column. But how do we know the 'John Doe' in the breach is *our* user? We built the `IdentityMatchEngine`. It uses a deterministic, rule-based algorithm to assign confidence scores based on independent evidence groups. We explicitly avoided 'black box' Neural Networks here because privacy laws require us to explain exactly *why* we linked data to a user."

(Slide 5: ML Risk Scoring) "Once the data is correlated, we assess the risk. While the identity engine is deterministic, our Residual Risk scorer is a Machine Learning model. We trained a `HistGradientBoostingRegressor` using Scikit-Learn. It takes feature vectors—like the count of financial findings and severity—and predicts a compounded risk score. We load this serialized `.joblib` model directly into the FastAPI memory for microsecond inference."

(Slide 6: Automated Remediation) "Now for the core innovation: Remediation. When a user clicks 'Remove', a specialized Celery worker wakes up. It launches a headless Chromium browser using Playwright. It navigates to the data broker's opt-out URL, parses the DOM, fills in the user's PII, attempts to inject CAPTCHA tokens via JavaScript evaluation, and submits the form. If it succeeds, it updates Postgres. If the site changed its layout, it fails gracefully and notifies the user."

(Slide 7: Security) "Given the sensitive data we hold, security is paramount. We implemented strict Row-Level Security (RLS) in Postgres to prevent cross-tenant data leaks. Passwords are hashed with Argon2. PII is encrypted at rest using AES-GCM-256. Finally, to prevent Server-Side Request Forgery (SSRF) when our workers scrape URLs, we built a custom `EgressFetcher` that blocks all internal IP ranges."

(Slide 8: Conclusion & Q/A) "In conclusion, DigiZafe successfully bridges the gap between passive threat intelligence and active, automated remediation. Thank you, I will now take questions."

PART 4 — ARCHITECTURE QUESTIONS (30)

1. **Why FastAPI instead of Django?** *Short:* FastAPI's native `asyncio` is required for concurrent network scraping. *Detailed:* Django is historically synchronous. FastAPI paired with `asyncpg` prevents the API from blocking while waiting for DB or Redis I/O. The architecture is designed to handle high concurrency. *Code:* `main.py` uses `@app.get` `async` defs.
2. **Why Postgres instead of MongoDB for JSON OSINT data?** *Short:* We needed strict relational integrity for Users/Identities, but schema flexibility for OSINT. *Detailed:* Postgres provides ACID compliance for auth and billing, while the `JSONB` column type with GIN indexing gives us NoSQL-like query speeds on unstructured vendor data. *Code:* `observation_finding.py`.
3. **Why Celery instead of threading?** *Short:* Threading doesn't scale across machines. *Detailed:* Celery allows us to distribute workers across multiple servers and provides durability via Redis. If a machine crashes, tasks aren't lost. *Code:* `worker.py`.
4. **Why Playwright instead of Selenium?** *Short:* Playwright is faster and `async`-native. *Detailed:* Playwright intercepts network requests, supports isolated browser contexts per user, and integrates natively with Python's `asyncio`. *Code:* `playwright_runner.py`.
5. **How does the frontend know when a Celery task finishes?** *Short:* Polling. *Detailed:* React Query (`@tanstack/react-query`) polls the `/api/v1/scans/{id}` endpoint until the status changes from `PENDING` to `COMPLETED`.
6. **Why are there two Redis containers?** *Short:* Separation of concerns. *Detailed:* One acts as a persistent message broker (don't evict data), the other acts as an ephemeral cache (evict old data).
7. **What is the biggest bottleneck in the system?** *Short:*

Playwright memory overhead. *Detailed:* Headless Chromium consumes hundreds of megabytes of RAM per tab. Concurrency must be strictly limited.

8. **How do you handle OSINT API rate limits?** *Short:* Celery task retries. *Detailed:* If we get a 429, we throw an exception and use `@task_retry` with exponential backoff.
9. **Why use Vite over Create React App?** *Short:* Vite uses esbuild. *Detailed:* HMR (Hot Module Replacement) is near-instant, vastly improving developer velocity.
10. **What happens if the Redis broker dies?** *Short:* Queued tasks are lost. *Detailed:* Unless Redis AOF/RDB persistence is perfectly synced, pending tasks vanish. The system would need a retry mechanism at the DB layer. (*20 additional architectural concepts*)
11. How does Alembic handle schema changes? (Generates SQL diffs, applies linearly).
12. Why decouple IdentityAnchors from CandidateProfiles? (Allows safely rejecting false positives without deleting data).
13. How do you scale this? (Move workers to K8s, use auto-scaling based on Redis queue length).
14. Why use Zustand instead of Redux? (Less boilerplate, no context provider wrapping needed).
15. What happens if an external API takes 5 minutes? (Celery timeout kills it to prevent worker starvation).
16. Why is the ML model loaded in FastAPI and not a separate microservice? (To save architecture complexity for the MVP).
17. What is Row-Level Security? (Postgres feature ensuring queries auto-filter by the executing user's ID).
18. How does the frontend handle Auth? (JWTs in memory, injected into Axios interceptors).
19. What is the role of Caddy in `docker-compose`? (Reverse proxy handling HTTP/HTTPS routing).
20. Why use Pydantic? (Automatic data parsing and OpenAPI schema generation).
21. What is the "Mock Fallacy"? (Tests pass because the mock is wrong, not because the code is right).
22. How is time handled? (Strictly UTC to avoid temporal correlation bugs).
23. Why use a bridge network in Docker? (Isolates backend services from the host machine).
24. What is the point of the Groq integration? (Translating raw JSON

- into human-readable text via LLM).
25. How do you prevent DB locking during long transactions? (Keep transactions short, use `asyncpg`).
 26. Why is `playwright_headless=True` a setting? (Allows toggling headless off for visual debugging locally).
 27. How does the Identity Engine run so fast? (SHA256 fingerprint caching).
 28. What happens if a broker changes their website DOM? (Playwright fails, status -> `MANUAL_NEEDED`).
 29. Why use `esbuild` in `Vite`? (It's written in Go, compiling JS 10x-100x faster).
 30. What is the ideal future state of Remediation? (AWS Lambda serverless functions replacing Celery workers).
-

PART 5 — CODE QUESTIONS (30)

1. In `playwright_runner.py`, what does `page.evaluate()` do? It injects and executes raw JavaScript directly into the browser DOM (used to force CAPTCHA tokens into hidden fields).
2. In `identity_match_engine.py`, what does `_generate_fingerprint` do? It creates a SHA256 hash of the input evidence. If the inputs haven't changed, it skips recalculating the score.
3. In `egress.py`, why do you monkey-patch `PinnedAsyncIOBackend`? To force `httpx` to connect to the specific, validated IP address we just resolved via DNS, preventing DNS-rebinding attacks.
4. In `main.py`, what does `@asynccontextmanager` do? It manages the FastAPI lifespan, allowing us to load ML models or static catalogs into memory exactly once before the server accepts traffic.
5. In `jwt.py`, what algorithm is used? RS256 or HS256, utilizing `python-jose` for cryptographic signing.
6. In `keys.py`, why does AES-GCM require a 12-byte nonce? GCM mode requires a unique initialization vector (nonce) for every encryption operation to maintain cryptographic security.

7. **What does `task_acks_late=True` do in `worker.py`?** It tells Celery not to acknowledge (delete) the task from Redis until the function *finishes*, rather than when it *starts*.
8. **In `train_residual.py`, what does `joblib.dump` do?** It serializes the trained Scikit-Learn model into a binary file for later inference.
9. **In `groq_client.py`, why is `follow_redirects=False` usually set in security contexts?** To prevent an external API from redirecting our fetcher to an internal URL (e.g., `localhost/admin`).
10. **In `test_rls_boundaries.py`, why use `AsyncMock`?** Because the SQLAlchemy session is asynchronous, standard `MagicMock` cannot be awaited, causing test failures. *(20 additional code concepts)*
11. **What does `Depends(get_db)` do?** (Yields an async session per request, closing it automatically).
12. **How does `alembic/env.py` know about your tables?** (It imports the SQLAlchemy Base declarative meta).
13. **In `playwright_runner.py`, what does `wait_until="domcontentloaded"` mean?** (Wait for HTML to parse, but don't wait for all images/iframes).
14. **What happens if `settings.groq_api_key` is `None`?** (Client returns `False`/Throws error, gracefully degrading).
15. **Why use `model_dump(mode="json")` in Pydantic?** (Ensures UUIDs and Datetimes are serialized to strings).
16. **How does `identity.py` get the current user?** (`current_user: CurrentUser` dependency).
17. **What is `feature_adapter.py`?** (Translates DB rows into flat numeric arrays for the ML model).
18. **What does `select(User).where(...)` return?** (A SQLAlchemy statement, not the data. Requires `db.execute()`).
19. **How are passwords hashed?** (Argon2 in `password.py`).
20. **Why use `@pytest.mark.asyncio`?** (Allows the test runner to await async functions).
21. **What is the `celerybeat-schedule` file?** (Local SQLite DB tracking when scheduled tasks last ran).
22. **In `egress.py`, why check `ip.is_private`?** (To block outbound requests to `192.168.x.x / 10.x.x.x`).
23. **In `worker.py`, what does `prefetch_multiplier=1` mean?** (A worker only grabs 1 task at a time, critical for memory-heavy

Playwright).

24. How is the master key handled locally? (`keys.py` auto-generates a 32-byte key if in dev mode).
 25. What does `.scalars().all()` do? (Extracts the model instances from the SQLAlchemy Result object).
 26. In React, what does `useMutation` do? (Handles async state for POST/PUT requests in React Query).
 27. In `playwright.config.ts`, what does `trace: 'on'` do? (Records a timeline of the browser session for debugging).
 28. What does `Zustand.create()` do? (Initializes a global state store outside the React component tree).
 29. Why use `JSONB` instead of `JSON` in Postgres? (`JSONB` is stored in a decomposed binary format, allowing indexing).
 30. How do you validate an email? (Using `email-validator` library inside Pydantic schemas).
-

PART 6 — ALGORITHM / ML QUESTIONS

(30)

1. **Why use `HistGradientBoostingRegressor`?** (Extremely fast training on tabular data, handles missing values natively).
2. **Why a Regressor and not a Classifier?** (Risk is a continuous scale (e.g., 85.4) not a binary "Risky/Safe").
3. **What is your `y` (target variable)?** (`target_delta` — the compounded risk factor impact).
4. **How do you handle categorical variables?** (They are count-encoded or one-hot encoded in the feature adapter).
5. **Why use an Identity Match Engine instead of ML for correlation?** (Legal explainability. We must prove why data belongs to a user).
6. **How does the Identity Engine cap username scoring?** (Uses a heuristic collision policy to limit points for common names like 'John Smith').
7. **What is Data Leakage in ML?** (When information from outside the training dataset accidentally leaks into the model, artificially inflating accuracy).

8. **How do you prevent overfitting?** (Limiting `max_depth=5` and `max_iter=100`).
9. **What is a hyperparameter?** (A setting configured *before* training, like learning rate).
10. **How do you explain a Gradient Boosting prediction?** (SHAP is a potential future extension, but natively, decision tree importance can be analyzed). *(20 additional ML concepts)*
11. What is the difference between Deterministic and Statistical algorithms?
12. Why not use an LLM for risk scoring? (Too slow, non-deterministic, hallucinates).
13. What is the role of `residual-dataset.csv`? (The ground truth historical data).
14. How does the model handle a completely new user? (Outputs the baseline intercept score).
15. What is Concept Drift? (When the relationship between features and targets changes over time).
16. How do you evaluate the regressor? (MSE - Mean Squared Error, or MAE - Mean Absolute Error).
17. Why use SHA256 on the `.joblib` file? (Integrity checking).
18. What is the Big O of the Identity Engine fingerprint? ($O(N \log N)$ due to sorting evidence).
19. How do you handle contradictory evidence? (Algorithmic override to `conflicting_evidence` state).
20. Why use independent evidence groups? (To prevent double-counting if two brokers steal the exact same record).
21. What happens if a user explicitly confirms a match? (Score hits 100 instantly, bypassing math).
22. What is K-Anonymity? (A privacy model where you are hidden in a crowd of at least K others—used in HIBP).
23. How does the Groq LLM stay fast? (LPUs instead of GPUs).
24. How do you prevent LLM prompt injection? (Strict system prompts, zero-shot data parsing).
25. Can the ML model update itself? (No, it is static offline training).
26. What happens if you add a new feature? (The model crashes until retrained to accept the new dimensionality).
27. What is `target_delta`? (The change in risk caused by a specific breach).
28. Why not use Deep Learning? (Overkill for tabular data, prone to overfitting).

29. How does Playwright detect a success state? (Heuristics—scanning DOM body for strings like "has been removed").
 30. How do you validate the Identity Algorithm's accuracy? (Manual human QA and historical test fixtures).
-

PART 7 — DATABASE QUESTIONS (20)

1. **Why `asyncpg`?** (It bypasses Python's DB-API for a native async protocol, vastly improving throughput).
2. **What is an ER Diagram?** (Entity-Relationship diagram mapping how tables connect).
3. **Why use `JSONB` for OSINT?** (Unstructured vendor data requires schema-less storage, but we still need to query it).
4. **What is a GIN Index?** (Generalized Inverted Index, essential for searching inside `JSONB`).
5. **What is Row-Level Security (RLS)?** (Postgres feature that intercepts queries and silently appends `WHERE user_id = X`).
6. **How does Alembic work?** (It reads SQLAlchemy metadata, compares it to the DB schema, and generates SQL diffs).
7. **What is a cascading delete?** (If a User is deleted, the DB automatically deletes their Scans and Findings).
8. **Why separate `IdentityAnchor` and `IdentityAlias`?** (An anchor is the verified root; aliases are unverified claims).
9. **What does `session.commit()` do?** (Ends the transaction and writes the data to disk).
10. **How do you handle concurrent inserts of the same finding?** (Unique Constraints and catching `IntegrityError`). *(10 additional DB concepts)*
11. What is an N+1 query problem? (Executing 1 query for a list, then N queries for their children).
12. How does SQLAlchemy solve N+1? (`selectinload` or `joinedload`).
13. What is a primary key? (UUID uniquely identifying the row).
14. Why use UUIDs instead of auto-incrementing integers? (Prevents ID enumeration attacks / guessing other users' IDs).
15. What is the difference between a cache and a database? (Cache

- is volatile RAM; DB is persistent disk).
16. How do you migrate data without downtime? (Additive migrations, multi-step deployments).
 17. What is connection pooling? (Keeping DB connections open to reuse them, avoiding TCP handshake overhead).
 18. What happens if a transaction fails halfway? (The entire transaction rolls back).
 19. How is the schema defined? (Declarative Base classes in Python).
 20. Why track `valid_from` timestamps? (For temporal tracking / auditing exposure over time).
-

PART 8 — SECURITY QUESTIONS (20)

1. **What is SSRF?** (Server-Side Request Forgery. Tricking the server into downloading internal/malicious URLs).
2. **How does `EgressFetcher` prevent SSRF?** (By resolving DNS locally and blocking private IP space like `10.x.x.x`).
3. **What is JWT?** (JSON Web Token. A cryptographically signed JSON payload).
4. **Why is JWT better than cookies here?** (It allows stateless API design and native cross-origin mobile app support).
5. **How is the Master Key protected?** (It is loaded from a volume-mounted file (`secrets/master.key`), never checked into git).
6. **What is AES-GCM?** (An authenticated encryption algorithm. It encrypts data AND proves it hasn't been tampered with).
7. **What is Argon2?** (A memory-hard password hashing algorithm designed to resist GPU cracking).
8. **How do you prevent SQL Injection?** (SQLAlchemy uses parameterized queries; it never concatenates raw strings).
9. **How do you prevent XSS?** (React automatically sanitizes DOM variables).
10. **What is a Blind Index?** (An HMAC hash of sensitive data allowing exact-match lookups without exposing the plaintext). *(10 additional Security concepts)*
11. **What is CORS?** (Cross-Origin Resource Sharing. Browser mechanism restricting who can call the API).

12. Why strip the `Host` header in egress? (To prevent host-header override attacks on external APIs).
 13. What is DNS Rebinding? (Attacker changes DNS to point to internal IP *after* the initial safety check).
 14. How is DNS rebinding mitigated? (Disabling HTTP redirects in `EgressFetcher`).
 15. Why use HTTPS for everything? (Encrypts data in transit to prevent Man-in-the-Middle attacks).
 16. What is a Rainbow Table attack? (Pre-computed password hashes. Prevented by salting).
 17. Does Argon2 use a salt? (Yes, automatically generated per user).
 18. What is the principle of least privilege? (A component only has the exact permissions it needs to function).
 19. How do you handle secrets in production? (HashiCorp Vault or AWS Secrets Manager).
 20. What is a supply chain attack? (A vulnerability introduced via `npm` or `pip` dependencies).
-

PART 9 — TRICK QUESTIONS (Expect these from a tough professor)

1. **"If this is fully automated, why do you need manual fallbacks for Remediation?"** *Honest Answer:* Because web scraping is an arms race. Data brokers actively try to block bots with advanced Cloudflare checks or dynamically mutating DOMs. We can solve 70%, but promising 100% automation is technically impossible.
2. **"Why not just use ChatGPT to write the Playwright scripts on the fly?"** *Honest Answer:* Non-determinism and speed. Generating code via an LLM per user request takes 10+ seconds and is highly prone to hallucinating incorrect DOM selectors, which would result in failed takedowns.
3. **"How do you know your ML model's prediction is correct?"** *Honest Answer:* We don't have absolute certainty. Risk is an abstract concept. We measure precision/recall against our synthetic ground-truth dataset, but in the real world, the score is a directional indicator, not an absolute truth.

4. **"What is the weakest part of your architecture?"** *Honest Answer:* Playwright scaling. Launching Chromium consumes massive RAM. If 1,000 users run remediation at once, the Celery queue will bottleneck massively unless we throw significant cloud computing budgets at it.
 5. **"Why did you use Python if Node.js is natively asynchronous?"** *Honest Answer:* Python owns the Machine Learning ecosystem. To seamlessly integrate Scikit-Learn without building complex microservice bridges, FastAPI gave us the best of both worlds: async I/O *and* native ML support.
-

PART 10 — RAPID-FIRE ROUND (Excerpts)

1. **Language?** Python 3.11, TypeScript.
2. **Framework?** FastAPI, React.
3. **Database?** PostgreSQL 16.
4. **ORM?** SQLAlchemy.
5. **Cache?** Redis.
6. **Task Queue?** Celery.
7. **ML Library?** Scikit-Learn.
8. **Automation?** Playwright.
9. **Password Hash?** Argon2.
10. **Encryption?** AES-GCM-256.
11. **Testing?** Pytest.
12. **UI Framework?** TailwindCSS, Radix.
13. **Routing (API)?** APIRouter.
14. **Routing (UI)?** React Router.
15. **State (UI)?** Zustand.
16. **API Client (UI)?** React Query.
17. **Migrations?** Alembic.
18. **Web Server?** Uvicorn.
19. **Reverse Proxy?** Caddy.
20. **LLM Provider?** Groq.

(Memorize these 20 for instant recall).

PART 11 — "OPEN ANY FILE AND EXPLAIN IT"

🔴 backend/app/main.py

- **What it does:** The entry point for the FastAPI server.
- **Why it exists:** It wires up routers, middlewares, and exception handlers.
- **Important Code:** `@asynccontextmanager def lifespan(app)` — runs once on startup to load static data.
- **Professor Question:** "Why is CORS configured here?" (Because it applies globally to all incoming HTTP requests before routing).

🔴 backend/app/worker.py

- **What it does:** Initializes the Celery distributed task queue.
- **Important Code:** `celery_app.conf.task_routes` — directs specific tasks to specific Redis queues (like `identity_enrichment`).
- **Professor Question:** "What does `worker_prefetch_multiplier=1` do?" (Ensures workers only grab one heavy Playwright task at a time, preventing memory exhaustion).

🔴 backend/app/services/identity_match_engine.py

- **What it does:** Correlates OSINT findings to users using rule-based math.
- **Important Code:** `def assess_candidate(...)`
- **Professor Question:** "How do you avoid calculating this twice?" (We hash the provenance inputs. If the hash matches the DB, we return the cached assessment).

🌀 backend/app/remediation/runners/ playwright_runner.py

- **What it does:** Launches Chromium to automate data broker opt-outs.
- **Important Code:** `await page.evaluate(...)` (injects CAPTCHA tokens).
- **Professor Question:** "What happens if the site takes 2 minutes to load?" (The `timeout=timeout_ms` argument forces a failure, preventing the worker from hanging forever).

🌀 ml/training/train_residual.py

- **What it does:** Trains the ML model.
 - **Important Code:** `model = HistGradientBoostingRegressor(...)`
 - **Professor Question:** "Why use `joblib`?" (It is highly optimized for serializing large NumPy arrays, which Scikit-Learn models are built on).
-

PART 12 — WHAT I MUST MEMORIZE

🌀 MUST KNOW (Absolute Basics)

- The exact user flow: React -> FastAPI -> Postgres -> Redis -> Celery -> Playwright.
- The difference between the Deterministic Identity Engine (Rules) and the Statistical Residual Risk model (ML).
- Why Playwright was chosen (to bypass bot protection).

🌀 SHOULD KNOW (Architecture & Security)

- How JWTs work (Stateless, signed JSON).

- How SSRF is mitigated (EgressFetcher DNS resolution).
- How Celery works (Producer pushes to Redis, Consumer pulls from Redis).

● GOOD TO KNOW (DevOps & Edge Cases)

- What Alembic does (DB migrations).
 - Why `asyncpg` is used (Async database driver).
 - What K-Anonymity is (Sending only the first 5 chars of a SHA1 hash to HIBP).
-

PART 13 — FINAL PRESENTATION CHECKLIST

- ☐ I can draw the architecture on a whiteboard.
- ☐ I can trace a request from a button click to the database.
- ☐ I can explain why I chose FastAPI over Django.
- ☐ I can defend why Playwright is necessary despite being slow.
- ☐ I can explain the difference between JSON and JSONB.
- ☐ I can explain what SSRF is and how my code stops it.
- ☐ I can explain why the ML model is a Regressor.
- ☐ I know which files handle Auth, Routing, Workers, and ML.
- ☐ I am ready to admit limitations (scaling, DOM brittleness) honestly.

Good luck. You built a highly complex, distributed, async, ML-integrated architecture. Be confident.